Smart Contract Security Audit Report

Table Of Contents

1 Executive Summary

On 2025.07.07, the SlowMist security team received the Elytro team's security audit application for Elytro Iterative Audit, developed the audit plan according to the agreement of both parties and the characteristics of the project, and finally issued the security audit report.

The SlowMist security team adopts the strategy of "white box lead, black, grey box assists" to conduct a complete security test on the project in the way closest to the real attack.

The test method information:

Test method	Description
Black box testing	Conduct security tests from an attacker's perspective externally.
Grey box testing	Conduct security testing on code modules through the scripting tool, observing the internal running status, mining weaknesses.
White box testing	Based on the open source code, non-open source code, to detect whether there are vulnerabilities in programs such as nodes, SDK, etc.

The vulnerability severity level information:

Level	Description
Critical	Critical severity vulnerabilities will have a significant impact on the security of the DeFi project, and it is strongly recommended to fix the critical vulnerabilities.
High	High severity vulnerabilities will affect the normal operation of the DeFi project. It is strongly recommended to fix high-risk vulnerabilities.
Medium	Medium severity vulnerability will affect the operation of the DeFi project. It is recommended to fix medium-risk vulnerabilities.
Low	Low severity vulnerabilities may affect the operation of the DeFi project in certain scenarios. It is suggested that the project team should evaluate and consider whether these vulnerabilities need to be fixed.
Weakness	There are safety risks theoretically, but it is extremely difficult to reproduce in engineering.
Suggestion	There are better practices for coding or architecture.

2 Audit Methodology

The security audit process of SlowMist security team for smart contract includes two steps:

- Smart contract codes are scanned/tested for commonly known and more specific vulnerabilities using automated analysis tools.
- Manual audit of the codes for security issues. The contracts are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities that was considered during the audit of the smart contract:

Serial Number	Audit Class	Audit Subclass
1	Overflow Audit	-
2	Reentrancy Attack Audit	-
3	Replay Attack Audit	-
4	Flashloan Attack Audit	-
5	Race Conditions Audit	Reordering Attack Audit
6	Permission Vulnerability Audit	Access Control Audit
		Excessive Authority Audit
7	Security Design Audit	External Module Safe Use Audit
		Compiler Version Security Audit
		Hard-coded Address Security Audit
		Fallback Function Safe Use Audit
		Show Coding Security Audit
		Function Return Value Security Audit
		External Call Function Security Audit

Serial Number	Audit Class	Audit Subclass
7	Security Design Audit	Block data Dependence Security Audit
		tx.origin Authentication Security Audit
8	Denial of Service Audit	-
9	Gas Optimization Audit	-
10	Design Logic Audit	-
11	Variable Coverage Vulnerability Audit	-
12	"False Top-up" Vulnerability Audit	-
13	Scoping and Declarations Audit	-
14	Malicious Event Log Audit	-
15	Arithmetic Accuracy Deviation Audit	-
16	Uninitialized Storage Pointer Audit	-

3 Project Overview

3.1 Project Introduction

The Elytro wallet is a rebranded version of the Soul wallet. This audit primarily covers the differences from the version of Soul wallet that was previously audited. The audit includes the newly added spending limit hook, as well as code modifications made to adapt to EntryPoint version 0.8.

3.2 Vulnerability Information

The following is the status of the vulnerabilities found in this audit:

NO	Title	Category	Level	Status
N1	Incorrect approve spend decoding	Design Logic Audit	High	Fixed
N2	Bypassable token spending checks	Design Logic Audit	Low	Fixed
N3	Limitations of Token Spend Checks	Design Logic Audit	Information	Acknowledged

4 Code Overview

4.1 Contracts Description

Audit Version:

https://github.com/Elytro-eth/Elytro-wallet-contract

commit: 3d64ccd3d0cd8228298cedb25af81eb042172c59

https://github.com/Elytro-eth/elytro-wallet-core

commit: 26d30a431b42c5f8241db58c6390443896a37077

Fixed Version:

https://github.com/CodeExplorer29/Elytro-wallet-contract

commit: db67a1a88eb8b7ae6a5b19a58f7bb726caa9dd70

Audit Scope:

NOTE: The Elytro wallet is a rebranded version of the Soul wallet. Therefore, the audit only covers the differences from the audited version of the Soul wallet, which mainly include the following areas:

```
./Elytro-wallet-contract/contracts/
├── factory
│   └── ElytroFactory.sol
├── hooks
│   └── spendLimit
│       └── DailyERC20SpendingLimitHook.sol
├── libraries
```

```
│   └── WebAuthn.sol
├── modules
│   └── upgrade
│       ├── IUpgradeModuleRegistry.sol
│       └── UpgradeModuleRegistry.sol
├── tools
│   └── ElytroInfoRecorder.sol
└── validator
    ├── ElytroDefaultValidator.sol
    └── libraries
        └── UserOpWithValidTimeRangeLib.sol
```

The main network address of the contract is as follows:

The code was not deployed to the mainnet.

4.2 Visibility Description

The SlowMist Security team analyzed the visibility of major contracts during the audit, the result as follows:

ElytroFactory			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	Ownable
_calcSalt	Private	-	-
createWallet	External	Can Modify State	-
proxyCode	External	-	-
_proxyCode	Private	-	-
getWalletAddress	Public	-	-
deposit	Public	Payable	-
withdrawTo	Public	Can Modify State	onlyOwner
addStake	External	Payable	onlyOwner
unlockStake	External	Can Modify State	onlyOwner
withdrawStake	External	Can Modify State	onlyOwner

DailyERC20SpendingLimitHook			
Function Name	Visibility	Mutability	Modifiers
supportsInterface	External	-	-
Init	External	Can Modify State	-
DeInit	External	Can Modify State	-
preIsValidSignatureHook	External	-	-
preUserOpValidationHook	External	Can Modify State	-
_decodeSpent	Private	-	-
_checkAndUpdateSpending	Internal	Can Modify State	-
_getDay	Private	-	-
initiateSetLimit	External	Can Modify State	-
applySetLimit	External	Can Modify State	-
cancelSetLimit	External	Can Modify State	-
_setInitialLimit	Internal	Can Modify State	-
getPendingLimit	External	-	-
getRemainingLimit	External	-	-
getCurrentLimit	External	-	-

WebAuthn			
Function Name	Visibility	Mutability	Modifiers
decodeP256Signature	Internal	-	-
decodeRS256Signature	Internal	-	-
recover_rs256	Internal	-	-
recover_p256	Internal	-	-

WebAuthn			
recover	Internal	-	-

UpgradeModuleRegistry			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	Ownable
latestVersion	External	-	-
addVersion	External	Can Modify State	onlyOwner
getVersionInfo	External	-	-
getVersionCount	External	-	-
getLatestModuleAddress	External	-	-
_isContract	Internal	-	-

ElytroInfoRecorder			
Function Name	Visibility	Mutability	Modifiers
recordKey	Private	-	-
recordData	External	Can Modify State	-
latestRecordAt	External	-	-

ElytroDefaultValidator			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
getDomainSeparatorV4	Public	-	-
validateUserOp	External	-	-
validateSignature	External	-	-

ElytroDefaultValidator			
_pack1271SignatureHash	Internal	-	-
_isOwner	Private	-	-
recover	Internal	-	-
supportsInterface	Public	-	-
Init	External	Can Modify State	-
DeInit	External	Can Modify State	-
getTypedDataHash	Public	-	-

UserOpWithValidTimeRangeLib			
Function Name	Visibility	Mutability	Modifiers
encode	Internal	-	-
hash	Internal	-	-

4.3 Vulnerability Summary

[N1] [High] Incorrect approve spend decoding

Category: Design Logic Audit

Content

In the DailyERC20SpendingLimitHook contract, the `_decodeSpent` function is used to decode relevant data regarding the wallet's spending of a specified token. If the target function called by the wallet is the `approve` function, `_decodeSpent` will decode and return the target token and the approved amount. Unfortunately, the target token is not assigned a value, which causes the `_decodeSpent` function to always return the zero address for the token when decoding an approve operation.

Code location: Elytro-wallet-contract/contracts/hooks/spendLimit/DailyERC20SpendingLimitHook.sol#L117-L123

```
function _decodeSpent(address target, uint256 value, bytes memory data)
    private
    view
    returns (address token, uint256 spent)
{
    if (value > 0) {
        ...
    }
    if (selector == IERC20.transfer.selector) {
        ...
    } else if (selector == IERC20.approve.selector) {
        // 0x095ea7b3   address  uint256
        // ____4_____|____32___|___32__
        assembly {
            spent := mload(add(data, 68)) // 32 + 4 +32
        }
        return (token, spent);
    } else if (selector == IERC20.transferFrom.selector) {
        ...
    }
    // no match
    return (address(0), 0);
}
```

Solution

It is recommended to assign the target address to the token variable when decoding approve operations.

Status

Fixed

[N2] [Low] Bypassable token spending checks

Category: Design Logic Audit

Content

In the DailyERC20SpendingLimitHook contract, the `_decodeSpent` function is used to decode relevant data related to a wallet spending a specified token. Currently, the decoding of spending actions only supports wallet executions that call the `transfer`, `approve`, and self-called `transferFrom` functions. However, it should be noted that the vast majority of popular on-chain tokens are based on versions of OpenZeppelin ERC20 prior to v5.0, all of which include the `increaseAllowance` function. Users can execute this function through their wallets to bypass the

spending check. Furthermore, if this wallet is used as an EIP7702 account, users may also bypass the check by leveraging certain token-specific permit or transferWithAuthorization features.

Code location: Elytro-wallet-contract/contracts/hooks/spendLimit/DailyERC20SpendingLimitHook.sol#L97

```
function _decodeSpent(address target, uint256 value, bytes memory data)
    private
    view
    returns (address token, uint256 spent)
{
    ...
}
```

Solution

Since the functionality implemented by each token varies, it is currently impossible to completely prevent spending checks from being bypassed. However, it is recommended to cover as many token spending functions as possible that are widely supported by the majority of tokens, such as the `increaseAllowance` function.

Status

Fixed

[N3] [Information] Limitations of Token Spend Checks

Category: Design Logic Audit

Content

As noted in N2, different tokens implement various functionalities, and the ability to directly or indirectly transfer tokens is not limited to the `transfer`, `approve`, or `transferFrom` functions. Tokens may be spent through other methods or token-specific features. Therefore, wallet owners should be aware that the restrictions provided by the DailyERC20SpendingLimitHook are limited in scope.

Code location: Elytro-wallet-contract/contracts/hooks/spendLimit/DailyERC20SpendingLimitHook.sol#L97

```
function _decodeSpent(address target, uint256 value, bytes memory data)
    private
    view
    returns (address token, uint256 spent)
{
```

```
        ...
    }
```

Solution

N/A

Status

Acknowledged; The project team has added notes in the code regarding the limitations of the spending checks.

5 Audit Result

Audit Number	Audit Team	Audit Date	Audit Result
0X002507080001	SlowMist Security Team	2025.07.07 - 2025.07.08	Passed

Summary conclusion: The SlowMist security team uses a manual and SlowMist team's analysis tool to audit the project. During the audit work, we found 1 high risk, 1 low risk vulnerability, and 1 information. All the findings were fixed or acknowledged. The code was not deployed to the mainnet.

6 Statement

SlowMist issues this report with reference to the facts that have occurred or existed before the issuance of this report, and only assumes corresponding responsibility based on these.

For the facts that occurred or existed after the issuance, SlowMist is not able to judge the security status of this project, and is not responsible for them. The security audit analysis and other contents of this report are based on the documents and materials provided to SlowMist by the information provider till the date of the insurance report (referred to as "provided information"). SlowMist assumes: The information provided is not missing, tampered with, deleted or concealed. If the information provided is missing, tampered with, deleted, concealed, or inconsistent with the actual situation, the SlowMist shall not be liable for any loss or adverse effect resulting therefrom. SlowMist only conducts the agreed security audit on the security situation of the project and issues this report. SlowMist is not responsible for the background and other conditions of the project.

Official Website

www.slowmist.com

E-mail

team@slowmist.com

Twitter

@SlowMist_Team

Github

https://github.com/slowmist